

innocent mistakes or from malicious attempts to break your programs. With the skills you'll learn in this chapter, you'll make your programs more applicable, usable, and stable.

Reading from a File

An incredible amount of data is available in text files. Text files can contain weather data, traffic data, socioeconomic data, literary works, and more. Reading from a file is particularly useful in data analysis applications, but it's also applicable to any situation in which you want to analyze or modify information stored in a file. For example, you can write a program that reads in the contents of a text file and rewrites the file with formatting that allows a browser to display it.

When you want to work with the information in a text file, the first step is to read the file into memory. You can then work through all of the file's contents at once or work through the contents line by line.

Reading the Contents of a File

To begin, we need a file with a few lines of text in it. Let's start with a file that contains *pi* to 30 decimal places, with 10 decimal places per line:

```
pi_digits.txt 3.1415926535
               8979323846
               2643383279
```

To try the following examples yourself, you can enter these lines in an editor and save the file as *pi_digits.txt*, or you can download the file from the book's resources through https://ehmatthes.github.io/pcc_3e. Save the file in the same directory where you'll store this chapter's programs.

Here's a program that opens this file, reads it, and prints the contents of the file to the screen:

```
file_reader.py from pathlib import Path

❶ path = Path('pi_digits.txt')
❷ contents = path.read_text()
print(contents)
```

To work with the contents of a file, we need to tell Python the path to the file. A *path* is the exact location of a file or folder on a system. Python provides a module called `pathlib` that makes it easier to work with files and directories, no matter which operating system you or your program's users are working with. A module that provides specific functionality like this is often called a *library*, hence the name `pathlib`.

We start by importing the `Path` class from `pathlib`. There's a lot you can do with a `Path` object that points to a file. For example, you can check that the file exists before working with it, read the file's contents, or write new data to the file. Here, we build a `Path` object representing the file *pi_digits.txt*, which we assign to the variable `path` ❶. Since this file is saved in the same